

Tiling Game

Barchin și Charos se joacă pe o grilă de $2N \times 2M$ celule pătrate unitare. Rândurile sunt numerotate de sus în jos de la 0 la $2N - 1$, iar coloanele sunt numerotate de la 0 la $2M - 1$ de la stânga la dreapta. Pentru $0 \leq i < 2N$ și $0 \leq j < 2M$, notăm celula din rândul i și coloana j cu (i, j) .

Barchin îi dă lui Charos $N \cdot M$ **blocuri**, unul câte unul. Fiecare bloc este un pătrat de 2×2 format din patru **piese** 1×1 . Barchin a colorat fiecare piesă a blocului în negru sau în alb, garantând în același timp că **cel puțin o piesă este albă**.

Charos trebuie să plaseze fiecare bloc pe grilă **imediat după ce îl primește**, fără a ști ce blocuri va primi mai târziu. Blocurile nu pot fi rotite. Fiecare bloc trebuie plasat în întregime în interiorul grilei, acoperind exact patru celule ale grilei. Mai mult, placa unitară din stânga sus a fiecărui bloc trebuie să acopere o celulă ale cărei coordonate de rând și coloană sunt ambele pare. Fiecare celulă a grilei poate fi acoperită de cel mult un bloc.

Barchin câștigă jocul dacă, după plasarea oricărui bloc, există un pătrat de 2×2 format din celule acoperite de patru plăcuțe negre. Formal, dacă celulele (a, b) , $(a + 1, b)$, $(a, b + 1)$, $(a + 1, b + 1)$ sunt toate acoperite de plăcuțe negre pentru un $0 \leq a < 2N - 1$ și $0 \leq b < 2M - 1$, atunci Barchin câștigă. Indicii a și b nu trebuie să fie pari.

Charos câștigă dacă plasează toate blocurile $N \cdot M$ fără ca Barchin să câștige vreodată. Rețineți că plasarea blocurilor $N \cdot M$ va acoperi complet grila.

Sarcina ta este să implementezi o strategie pentru ca Charos să câștige jocul. Se poate demonstra că, sub constrângerile date, Charos poate oricând plasa blocurile astfel încât să garanteze o victorie, indiferent de culorile blocurilor pe care le primește ulterior.

Detalii de implementare

Ar trebui să implementați două proceduri:

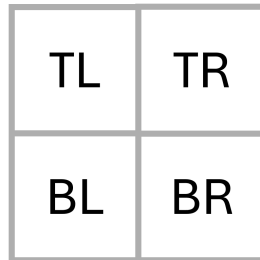
```
void init(int N, int M);
```

- N : jumătate din numărul de rânduri din grilă.
- M : jumătate din numărul de coloane din grilă.

- Procedura este apelată exact o singură dată per caz de test, la începutul execuției programului.

```
std::pair<int, int> receive_block(int TL, int TR, int BL, int BR);
```

- TL, TR, BL, BR : culorile plăcilor din stânga sus, dreapta sus, stânga jos și respectiv dreapta jos ale blocului curent, așa cum se arată în figura de mai jos. Fiecare valoare este fie 0 (alb), fie 1 (negru).
- Această procedură este apelată exact de $N \cdot M$ ori per caz de test, după apelul inițial al comenzii `init`.



Această procedură ar trebui să returneze o pereche de numere întregi (i, j) , unde i este coordonata rândului și j este coordonata coloanei celei în care ar trebui plasată placa din stânga sus a acestui bloc. Atât i cât și j trebuie să fie pare, iar regiunea 2×2 acoperită de bloc nu trebuie să se suprapună peste niciun bloc plasat anterior.

Dacă `receive_block` returnează vreodată o pereche care nu îndeplinește aceste cerințe sau dacă după plasarea blocului un pătrat de 2×2 de celule devine complet acoperit de plăci negre, evaluatorul va termina imediat programul, iar verdictul pentru cazul de testare va fi `Output isn't correct`.

Comportamentul evaluatorului **nu este adaptiv**. Aceasta înseamnă că secvența de blocuri pe care Barchin le dă lui Charos este fixă înainte de apelul `init`.

Restricții

Pentru fiecare bloc, fie S numărul de plăci negre dintre cele patru plăci ale sale. Adică, $S = TL + TR + BL + BR$.

- $1 \leq N, M \leq 100$
- $0 \leq S \leq 3$ pentru fiecare bloc.

Subtask-uri

Subtask	Scor	Restricții suplimentare
1	6	$S = 1$ pentru fiecare bloc și $N = 2$.
2	16	$S = 3$ pentru fiecare bloc. $N = M$, N este par, iar fiecare dintre cele patru colorări posibile ale blocurilor apare exact de $\frac{N^2}{4}$ ori.
3	10	$S = 1$ pentru fiecare bloc.
4	29	$S \leq 2$ pentru fiecare bloc.
5	39	Fără restricții suplimentare.

Exemplu

Să luăm în considerare un joc cu $N = 1$ și $M = 2$, deci grila are 2 rânduri și 4 coloane. Graderul apelează mai întâi:

```
init(1, 2)
```

Inițial, toate celulele sunt goale. Grila arată astfel:

	0	1	2	3
0				
1				

Există $N \cdot M = 2$ blocuri de plasat. Să presupunem că Barchin dă un bloc cu trei plăcuțe negre și o plăcuță albă în colțul din dreapta sus. Evaluatorul anunță:

```
receive_block(1, 0, 1, 1)
```

Charos decide să plaseze acest bloc în partea stângă a grilei returnând $(0, 0)$.

Grila arată acum astfel:

	0	1	2	3
0				
1				

Barchin dă apoi un alt bloc cu trei plăcuțe negre și o plăcuță albă în colțul din stânga sus:

```
receive_block(0, 1, 1, 1)
```

Singura celulă rămasă cu un rând par și o coloană pară care poate servi drept colț din stânga sus al unui bloc 2×2 este $(0, 2)$, deci Charos returnează $(0, 2)$. Grila se completează astfel:

	0	1	2	3
0				
1				

Niciun pătrat 2×2 nu este complet acoperit de plăcuțe negre, așa că Charos a plasat cu succes toate blocurile fără ca Barchin să câștige vreodată. Charos câștigă jocul.

Grader local

Format de intrare:

```
N M
TL[0] TR[0] BL[0] BR[0] TL[1] TR[1] BL[1] BR[1]
... TL[NM-1] TR[NM-1] BL[NM-1] BR[NM-1]
```

Format de ieşire:

```
R[0] C[0]  
R[1] C[1]...  
R[NM-1] C[NM-1]
```

Aici, $R[k]$ şi $C[k]$ sunt perechea de numere întregi returnată de al k -lea apel către `receive_block`.